Working Product & Development Report

Project: Explainable Market – AI-RPA Predictive Analysis Platform

Github Repository: https://github.com/ning-111/explainable_market

Team Members: Jose Manuel Cortes Ceron, Benedikt Darrel Joewono, Haining Yu, Aldrich Yuanto, Omar Ahmed, Ziyi Wang.

Date: 30 de Octubre de 2026

1. The App Working: Proof of Concept & Demo

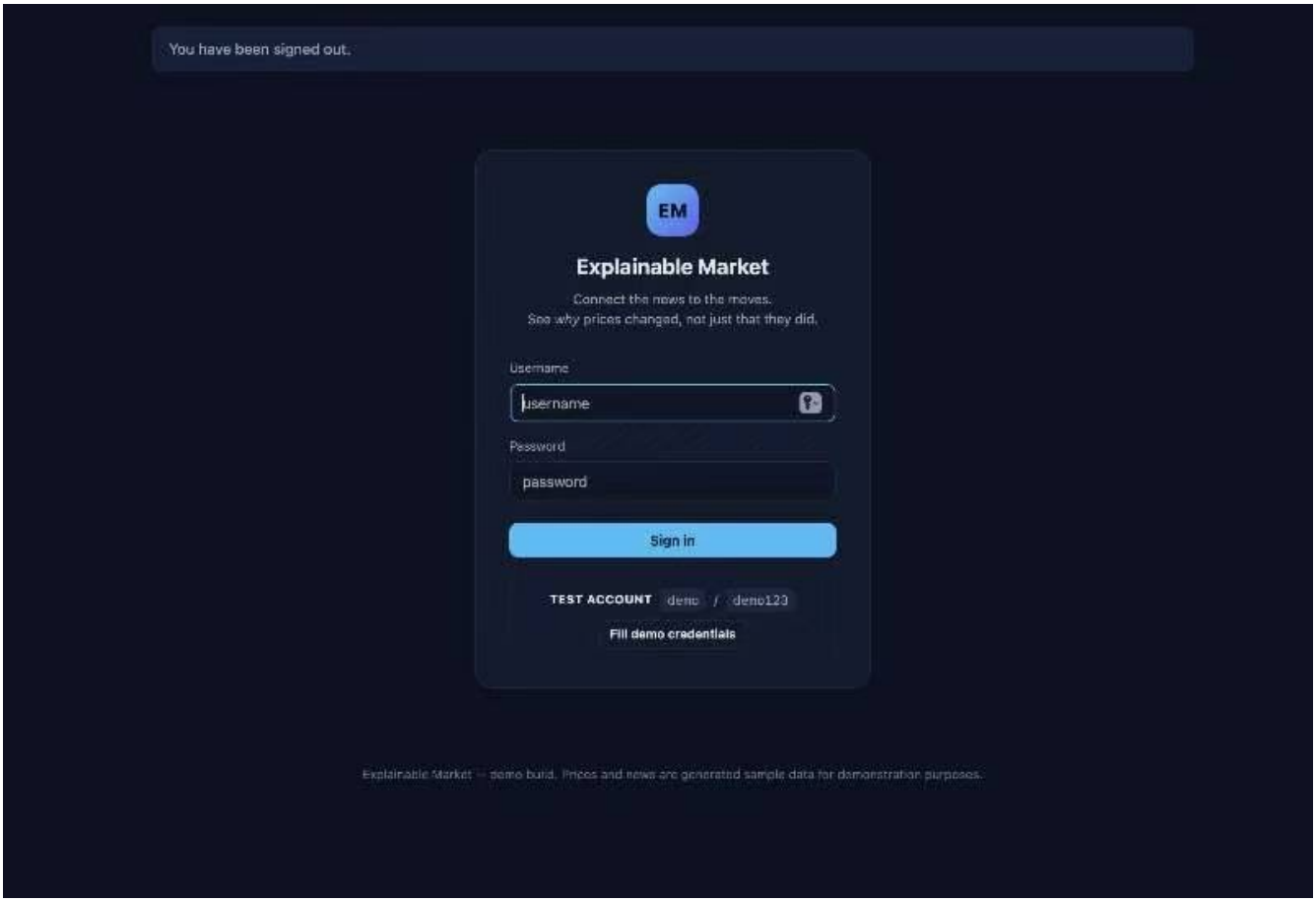
We have moved beyond the idea and prototype stage. We now have a functioning web application. The application is fully built, featuring a responsive dark-mode desktop interface, and is connected to a local **SQLite database** for data persistence (user authentication, watchlists, and generated news data).

Test Account Credentials (SQLite):

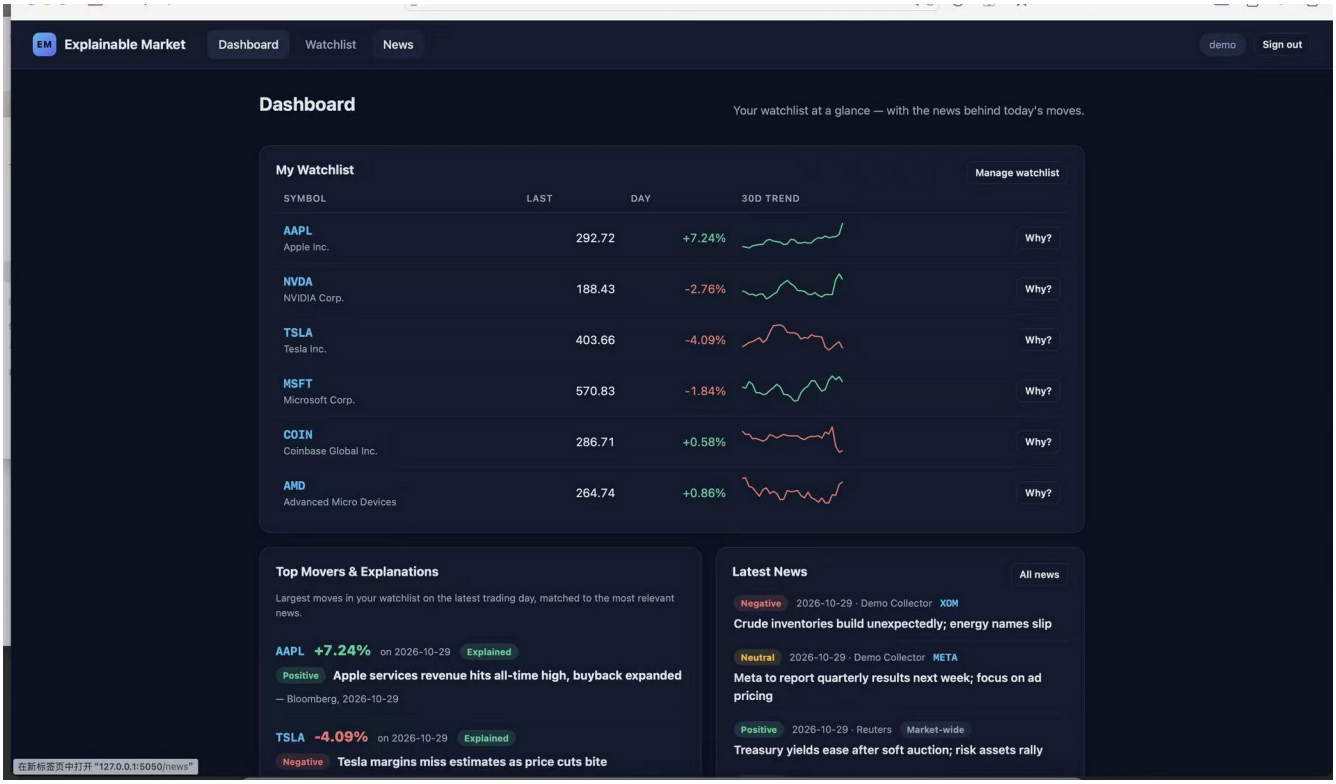
•Username: demo@user.com
•Password: d3m012!

App Screenshots (Real Usage):

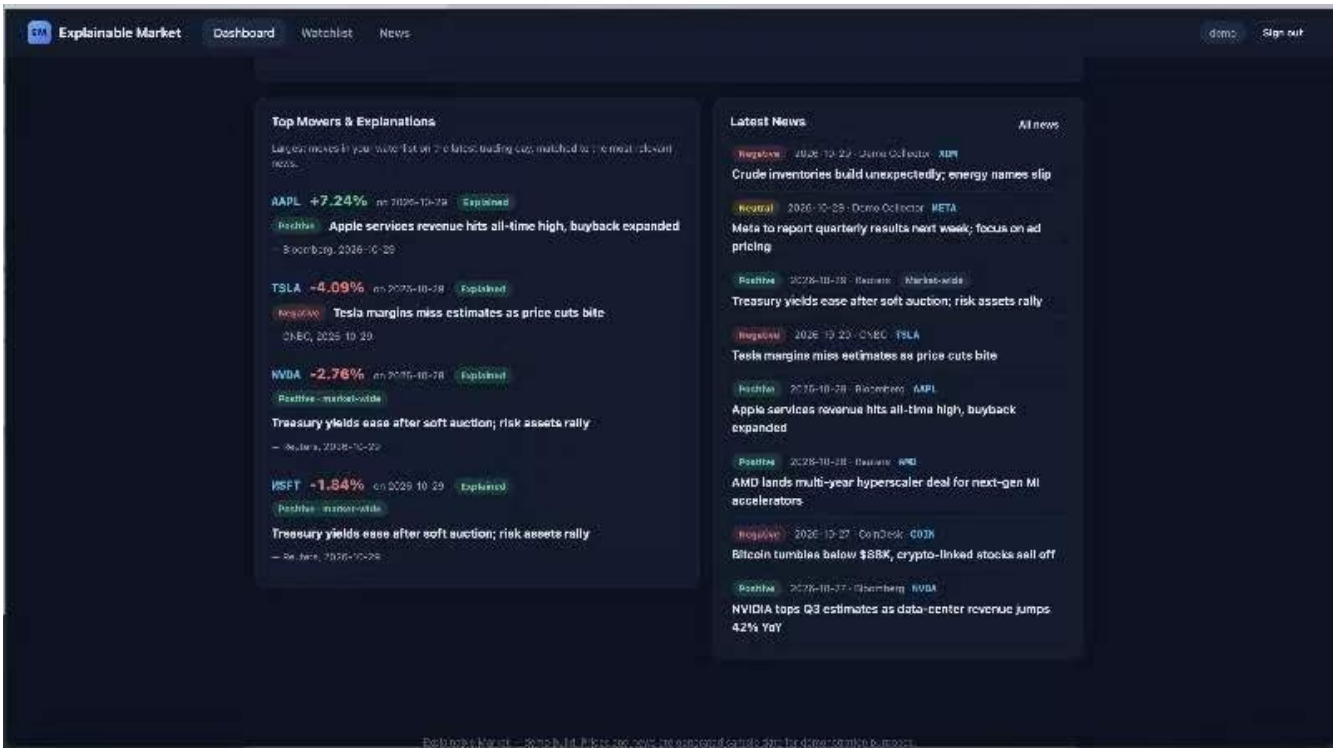
1.**Secure Login:** The user is logged out and presented with the clean, centered authentication screen.



2.**Dashboard Overview:** After logging in, the user sees "My Watchlist" populated with real (demo) data, including AAPL, NVDA, TSLA, etc., complete with 30-day trend sparklines and a "Why?" button for explainability.



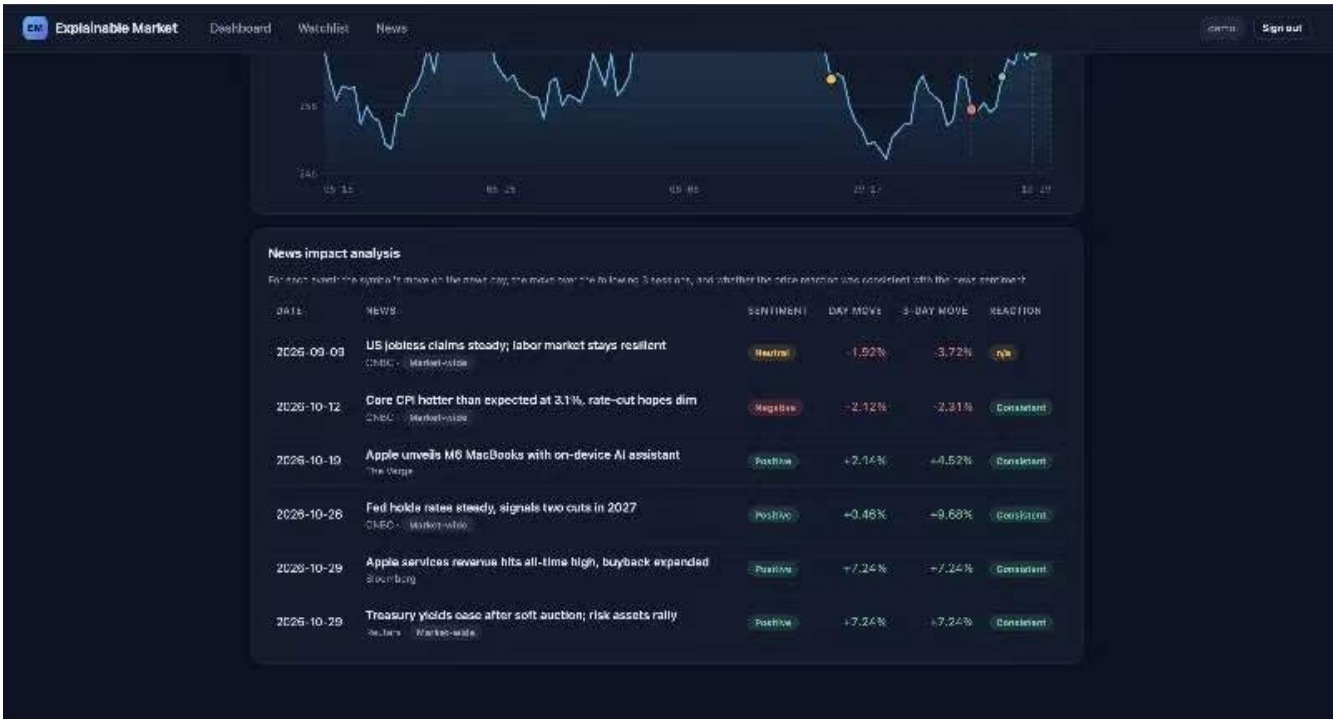
3.**Top Movers & Explanations:** The dashboard highlights the biggest price movements and links them to the specific news article that caused the shift (e.g., Apple +7.24% linked to revenue news).



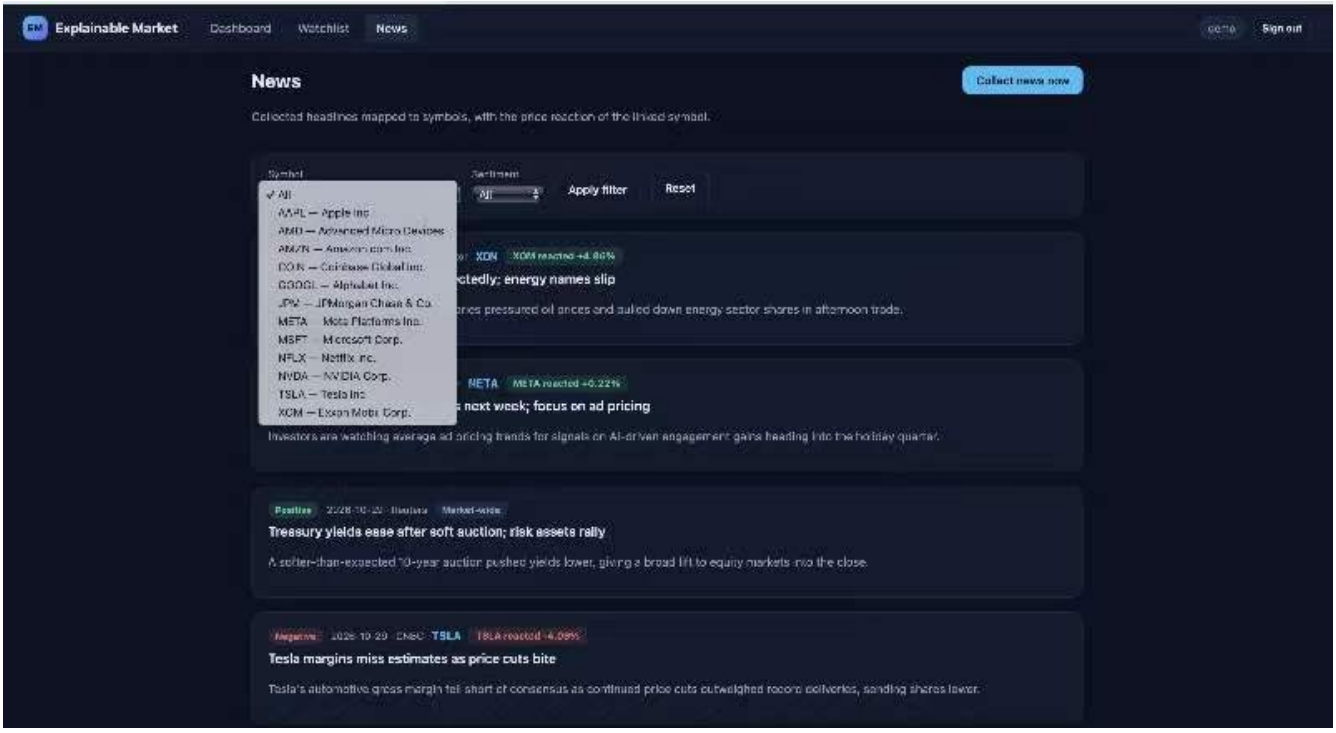
4.**Asset Detail (AAPL):** Clicking on a stock shows a detailed price timeline with historical data points and AI anomaly markers.



5.**News Impact Analysis:** The News tab shows how the AI sentiment engine has mapped recent headlines to specific tickers and their subsequent price movements (1-day, 3-day, 7-day).



6.**Filterable News Feed:** Users can filter news by ticker (e.g., AAPL, AMD) and sentiment (Positive, Negative, Neutral).



2. Three Meaningful Features

Here is the breakdown of our three core features, detailing the input, processing, output, and current status.

Feature 1: "Explainable" Watchlist & Trend Visualization

•**Input:** The user views the main Dashboard.
•**What the app does:** The app queries the SQLite database to retrieve the user's saved tickers. It renders the current price, daily percentage change (color-coded green/red), and a 30-day sparkline chart. Crucially, it includes a **"Why?"** button next to each asset.
•**Output:** A clean, data-dense watchlist. Clicking "Why?" bridges the gap between raw data and explainability, linking the user to the news that justifies the movement.
•**Status: Done.**

Feature 2: Top Movers & AI News Correlation

•**Input:** The system pulls the day's largest percentage gainers/losers from the watchlist.
•**What the app does:** It cross-references these movements with the "Latest News" feed. The app tags the news with Sentiment (Positive/Negative) and specific Tickers.
•**Output:** The "Top Movers & Explanations" section. For example, it clearly shows AAPL +7.24% and directly below it tags the headline "Apple services revenue hits all-time high" as **Positive** and marks it as **Explained**.
•**Status: Done.**

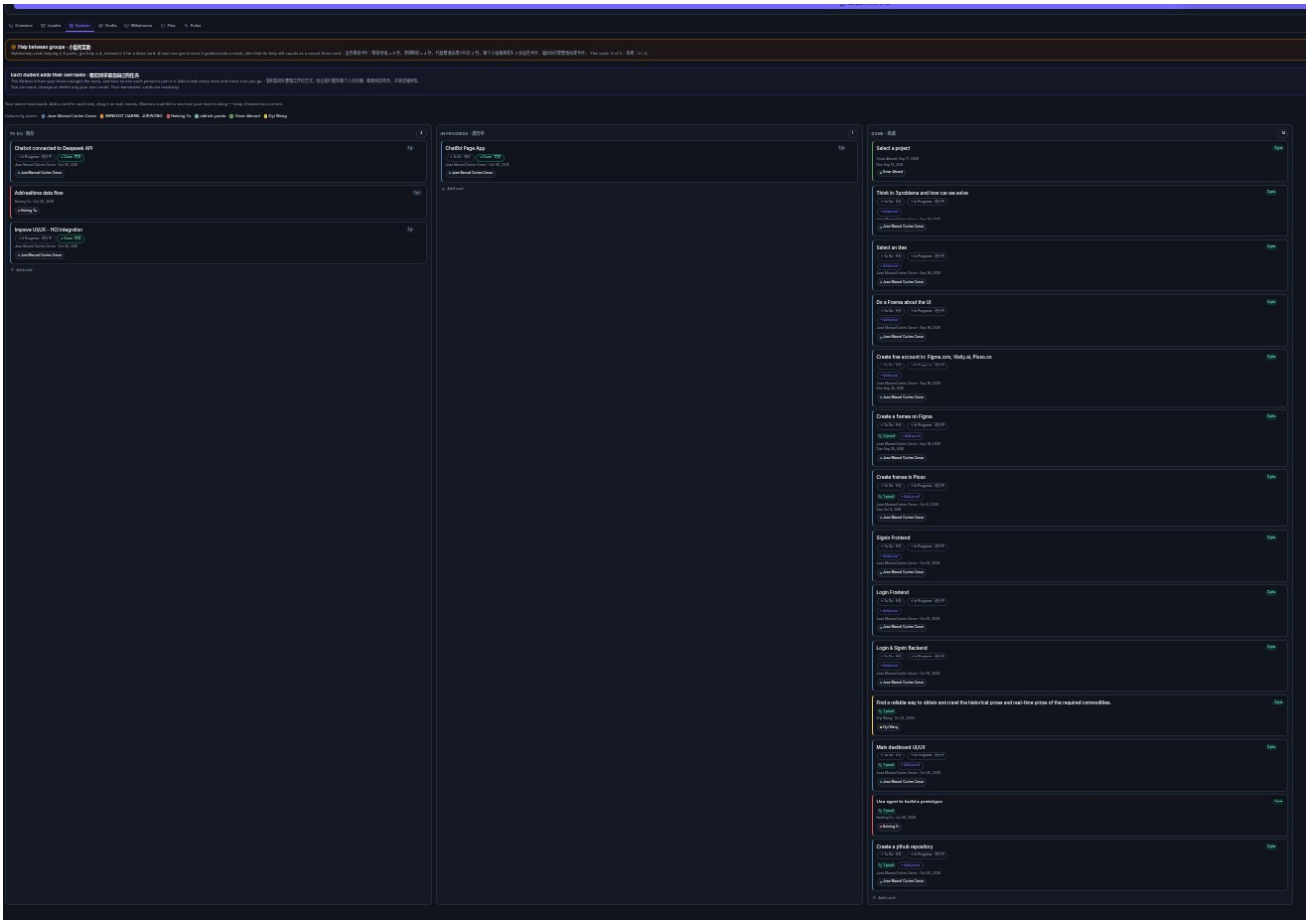
Feature 3: Deep-Dive Asset Timeline & Human-in-the-Loop Markers

•**Input:** The user clicks on a specific stock from the Watchlist.
•**What the app does:** The app retrieves historical price data and news events from SQLite. It plots a detailed price timeline and overlays markers (dots) for news events or AI anomalies.
•**Output:** The "Price & news timeline" view (e.g., for AAPL). It shows a "News impact analysis" table at the bottom, detailing the exact date, the news, its sentiment, and the 1-day/3-day/7-day price moves.
•**Status: Done.**

---

3. Kanban Evidence & Team Plan

We used the **UnoAnalyzer** platform to manage our development. Our Kanban board is updated weekly to ensure transparency and track individual contributions.



Kanban Evidence:

Agreed Plan with Mentor:

•**Week 1-2:** Define problems, select project, and create initial UI frames. (Owner: Jose, Due: Sep 18)
•**Week 3-4:** Develop Data Reception and Login/Signin backend integration with SQLite. (Owner: Jose, Due: Oct 16)
•**Week 5:** Implement Checkpoint 2. Build ChatBot Page App and connect to Deepseek API. (Owner: Jose, Due: Oct 30)
•**Week 6-8:** Integrate actual RPA scripts, replace SQLite sample data with real-time financial APIs, and deploy the app.

Current Blockers: We need to obtain API keys for a real-time financial data provider to replace the static sample data currently stored in SQLite.

4. Tech Stack Rationale

We chose our tech stack to balance rapid prototyping with the ability to scale into a real AI-RPA system.

•**Frontend: HTML/CSS/JavaScript** (Custom Dark Mode design system).

•Why: We designed the high-fidelity UI in Pixso, then translated it directly to code. We opted for a desktop-first web application because financial analysts require dense data visualization that is difficult to read on a small mobile screen.

•**Database: SQLite.**

•Why: SQLite allowed us to build a fully functioning, relational database without the overhead of setting up a cloud server. It perfectly handles the relationships between Users, Tickers, News, and Sentiments.

•**AI Integration: DeepSeek API.**

•Why: We are using DeepSeek to power the chatbot and sentiment analysis features. It offers high performance for logical reasoning and text processing at a very competitive cost compared to other LLM providers.

•**Backend & RPA (Next Phase): Python (FastAPI) + UiPath.**

•Why: Python is the industry standard for financial modeling. UiPath will handle the RPA side (navigating legacy systems), and LangChain will orchestrate the AI agents.

•**What we considered:** We considered using a NoSQL database (MongoDB) for flexibility, but financial data is highly relational. SQLite was the perfect stepping stone for this checkpoint.

•**Easier/Harder:** SQLite made data persistence incredibly easy. The hardest part was designing the UI to handle dense tabular data without overwhelming the user, ensuring the "Explainability" factor remained front and center.

5. Building Reflection & Plan to Finish

What was difficult:

The hardest part was designing the "News Impact Analysis" table. We had to ensure that the user could easily trace a price movement (e.g., a 7.24% jump) back to a specific news headline and understand the sentiment (Positive/Negative). Making this readable in a

dark-mode UI required a lot of iteration on contrast and spacing. Additionally, connecting the Chatbot to the DeepSeek API required careful handling of asynchronous requests.

What changed:
We originally planned to build a mobile-first app for on-the-go analysts. However, during development, we realized that the core value of "Explainable Market" relies on comparing tables of data, looking at detailed 30-day timelines, and reading long-form news. We pivoted to a **desktop-first web application** to accommodate this data density.

What we learned:
We learned that in fintech, "Explainability" is just as important as "Automation." It's not enough for the AI to show that a stock went up; the user needs to see why it happened. Our design journey taught us to prioritize trust-building UI elements over purely aesthetic ones.

What is left to do:

Task	Owner	Deadline
Ensure the complete code (Frontend/Backend/DeepSeek integration) is pushed to the GitHub repository.	Jose Manuel	Nov 1
Connect to real-time market data APIs (replace SQLite mock data).	Benedikt / Haining	Nov 10
Refine the AI Sentiment generation using the DeepSeek API in the backend.	Aldrich / Omar	Nov 15
Implement the RPA (UiPath) proof-of-concept for data extraction.	Ziyi / Jose	Nov 20
Final usability testing with 5 finance students.	Entire Team	Nov 25
Prepare the final presentation and demo video.	Entire Team	Nov 28